Systems Design Document: Selection of Cryptographic Techniques for the Privacy Preserving Data Analytics Engine (E-Privacy)

Dilmith D. L. S., Hansana O. G. V., Vidyananda K. H. K.

September 11, 2026

1 Introduction

This document identifies the specific cryptographic algorithms, modes, and parameters used to implement each component of the Privacy Preserving Data Analytics Engine (PPDAE) specified in the systems requirements specification. Rather than treating "encryption" as a single monolithic control, this design follows the standard three-state model of data protection – *at rest*, *in transit*, and *in use* – and selects a distinct, purpose-fit technique for each, since a single cipher cannot satisfy all three simultaneously (in particular, the "in use" state requires an additively homomorphic scheme, not a conventional block cipher).

1.1 Assumptions

B1. All parties (Data Owner client and cloud compute nodes) can execute standard, well-reviewed cryptographic libraries (e.g. OpenSSL, libsodium, python-phe) rather than custom implementations of primitives.

B2. The Data Owner's private Paillier key and any symmetric key-encryption keys are generated using a cryptographically secure random number generator (CSPRNG) seeded from the host operating system's entropy source.

B3. Clock synchronization (e.g. NTP) is available across components, since signed audit-log entries and TLS session validation depend on reasonably accurate timestamps.

B4. The 2048-bit Paillier key length is adopted as the default operating point, per the empirical latency/storage trade-off analysis in Section 4: it gives an ∼11x storage expansion and sub-5-minute aggregation latency for a 2M-record batch, versus 3072-bit's disproportionate latency cost.

B5. Where a technique below is described as protecting data "while in" a given state, that state boundary corresponds exactly to the trust-domain boundary shown in Figure 1.

2 Cryptographic Techniques by Data State

Figure 1 maps each data-flow segment of the Engine to its assigned cryptographic control.

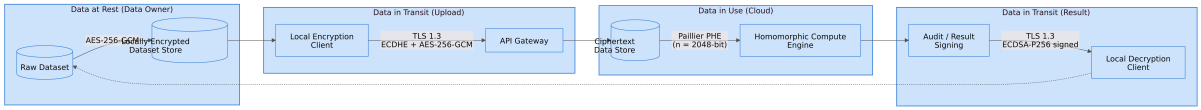


Figure 1: Cryptographic technique assigned to each data state and transition across the trust boundary.

3 Component-Level Cryptographic Specification

3.1 Data at Rest (Data Owner's Local Store)

The raw sensitive dataset, before it is ever homomorphically encrypted for outsourced computation, is protected using the **AES block cipher with a 256-bit key in GCM mode** (AES-256-GCM) to ensure confidentiality and integrity while the data resides on the Data Owner's local disk. GCM is chosen over CBC because it provides authenticated encryption (confidentiality *and* integrity/tamper-detection) in a single pass, which CBC alone does not. A random 96-bit nonce is generated per encryption operation and never reused with the same key, as required by GCM's security proof.

3.2 Key Management

- **Paillier keypair generation:** a probabilistic public/private keypair is generated locally by the Data Owner using two large random primes p, q of equal bit-length, giving a modulus $n = pq$ of the configured length (1024/2048/3072 bits, default 2048 per B4). The public key (n, g) is shared with the cloud engine; the private key (λ, μ) never leaves the Data Owner's trusted domain (per SRS requirement FR-1).
- **Key-encryption key (KEK):** the Paillier private key and the local AES data-encryption key are themselves encrypted at rest under a KEK derived from an administrator passphrase using **Argon2id** (memory-hard KDF, resistant to GPU/ASIC brute-force), with parameters $t = 3$ iterations, $m = 64$ MiB memory, $p = 4$ parallelism lanes.
- **Key rotation:** Paillier keypairs are rotated per analytics campaign (e.g. per reporting quarter); historical ciphertext under a retired key remains decryptable only until that key is securely destroyed, per the organization's data-retention policy.

3.3 Data in Transit

All network communication between the Data Owner's client and the cloud API Gateway, and between the cloud engine and the client on the return path, uses **TLS 1.3** with an ECDHE (Elliptic-Curve Diffie–Hellman Ephemeral) key exchange on curve `X25519` and the `TLS_AES_256_GCM_SHA384` cipher suite. TLS 1.3 is selected over 1.2 because it removes legacy negotiable weak cipher suites and mandates forward secrecy for every handshake,

so that compromise of a long-term key does not retroactively expose past ciphertext-upload sessions.

3.4 Data in Use (Homomorphic Computation)

Aggregation queries (sum, average, count) execute directly over ciphertext using the **Paillier cryptosystem**, a partially homomorphic public-key scheme with the additive property $E(m_1) \cdot E(m_2) = E(m_1 + m_2) \bmod n^2$ and the affine property $E(m)^k = E(km) \bmod n^2$. This is the only cryptographic control in the design that operates while data is being computed on, which is precisely the "Data-in-Use" gap identified as the central architectural flaw of conventional analytics pipelines. Per B4, the default modulus length is **2048 bits**, giving 112-bit-equivalent security while keeping ciphertext storage and analytics latency within the operational budget established empirically in Section 4.

3.5 Access Control and Authentication

API requests to the Gateway are authenticated using **JSON Web Tokens (JWT) signed with ECDSA on curve P-256** (ES256). Tokens are short-lived (15-minute expiry) and scoped to a specific role (data ingestion, query submission, audit read) to enforce least privilege. Service-to-service calls between the Gateway and the Homomorphic Compute Engine additionally use **mutual TLS (mTLS)**, so that only provisioned compute nodes – not an arbitrary caller holding a valid JWT – can submit ciphertext for aggregation.

3.6 Integrity of the Audit / Verifiable Computation Log

Each entry appended to the Verifiable Computation Log (operation type, input ciphertext hashes, timestamp) is hashed with **SHA-256** and chained to the previous entry's hash (a hash chain), so that any retroactive edit or deletion is detectable. The chain's current head, together with the computation result, is signed using **ECDSA-P256**, giving any auditor a way to verify both that the log has not been tampered with and that the signature originates from the legitimate compute engine. This mechanism underpins the individual and universal verifiability properties detailed separately in the verifiability evaluation deliverable.

4 Empirical Justification of Cryptosystem and Key-Length Selection

To ground the choice of Paillier at a 2048-bit modulus (assumption B4 and the "Data in Use" specification above) in evidence rather than assertion, the Feasibility and Computation Overhead studies benchmarked four candidate approaches – **Plaintext** (baseline), **RSA** (decrypt-compute-re-encrypt), **PHE / Paillier** (additive homomorphism), and **FHE / TenSEAL CKKS** (arbitrary homomorphism) – across key lengths of 1024, 2048, and 3072 bits, extrapolated from a 1,000-record sample to a full 2,000,000-record / 94.76 MB financial dataset (a 2000x scale factor).

4.1 Storage Footprint

Figure 2 shows that FHE ciphertext expansion is catastrophic for tabular aggregation workloads (up to 617 GB for a 94.76 MB source file, a ∼6,600x expansion), while PHE remains within a manageable ∼11–16x expansion across all tested key lengths.

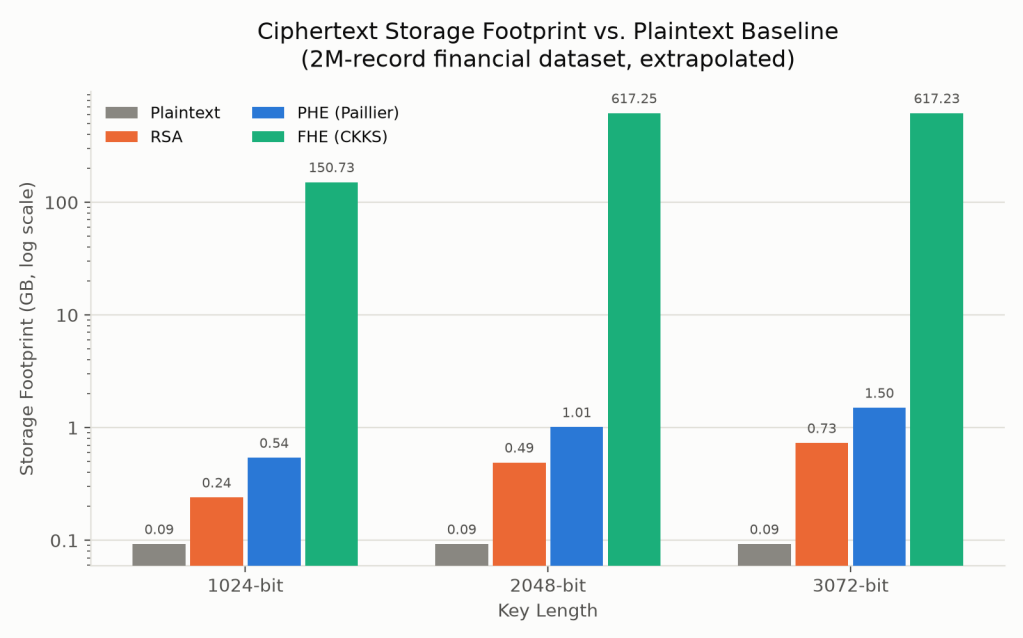


Figure 2: Ciphertext storage footprint by cryptographic scheme and key length (log scale).

4.2 Encryption Latency

Figure 3 shows the full-dataset encryption latency. PHE encryption is the slowest to *ingest* at higher key lengths (112 hours at 3072-bit), which is why the default configuration (B4) targets 2048-bit keys and a parallelized/pipelined ingestion process rather than blocking on full-dataset encryption before any query can run.

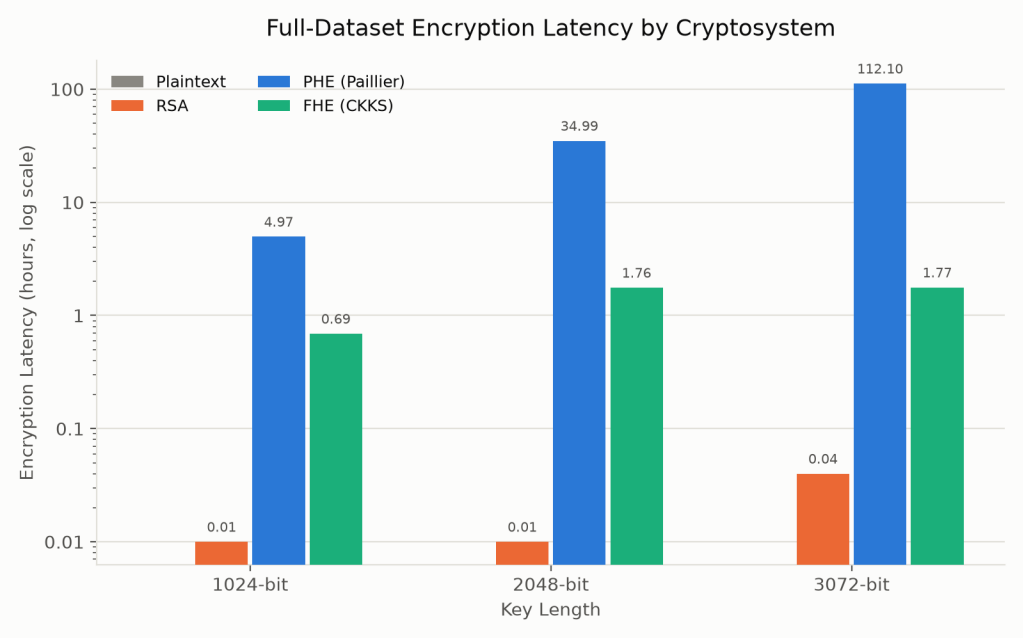


Figure 3: Full-dataset encryption latency by cryptographic scheme and key length (log scale).

4.3 Analytics (Query) Latency

Figure 4 shows the latency of the actual sum/average computation on already-encrypted data – the metric that dominates user-perceived query responsiveness. PHE outperforms both RSA's decrypt-compute-re-encrypt pipeline (which additionally violates the Data-in-Use confidentiality requirement, since it requires server-side plaintext exposure) and FHE at every key length tested.

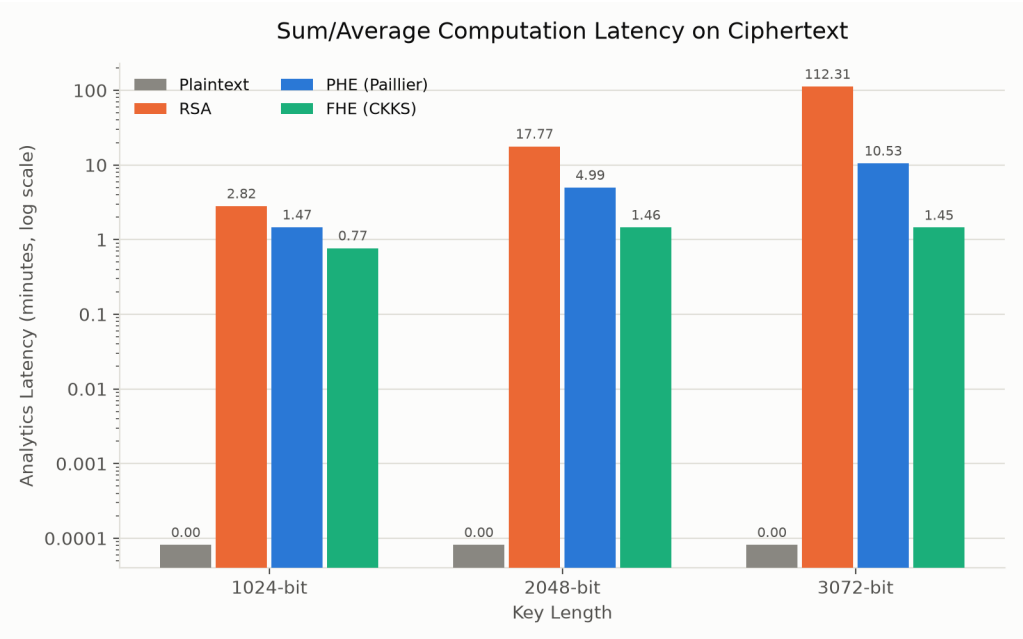


Figure 4: Ciphertext sum/average computation latency by cryptographic scheme and key length (log scale).

4.4 Trade-off Synthesis

Figure 5 plots storage footprint against analytics latency for the three encrypted schemes. PHE occupies the most favorable region of the trade-off space – low storage expansion *and* low query latency – for additive aggregation workloads, whereas FHE's generality is paid for with orders-of-magnitude worse storage and no corresponding analytics-latency benefit for this workload class.

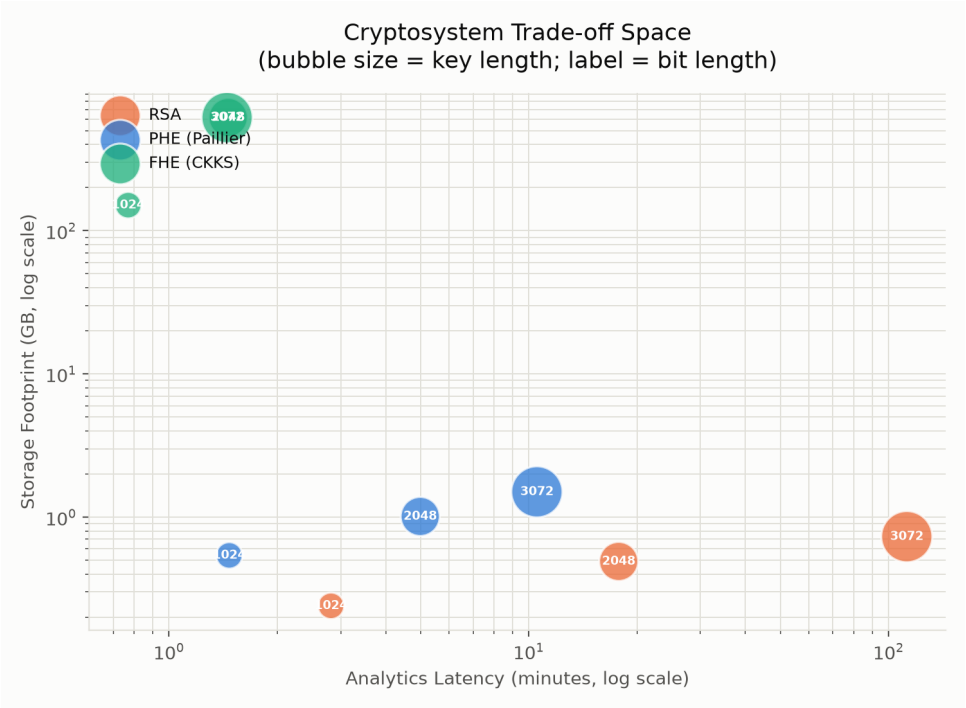


Figure 5: Storage footprint vs. analytics latency trade-off space across cryptosystems and key lengths.

4.5 Decision

Based on this empirical evidence, the Homomorphic Computation Engine is specified around Partially Homomorphic Encryption (Paillier) at a default 2048-bit key length, matching the industry-standard security margin while keeping storage expansion and analytics latency within operationally acceptable bounds. FHE (CKKS) is retained as a *pluggable future option* for workloads that require multiplicative operations between two ciphertexts, which Paillier does not support.

5 Summary of Cryptographic Parameters

Component	Technique	Parameters
Local dataset store	AES-GCM	256-bit key, 96-bit nonc
KEK derivation	Argon2id	t=3, m=64 MiB, p=4
Network transport	TLS 1.3 (ECDHE X25519)	`TLS_AES_256_GCM_SHA3`
Homomorphic aggregation	Paillier (PHE)	n = 2048-bit modulus

6 Conclusion

No single cryptographic primitive can secure the Engine end-to-end: symmetric AES-GCM protects data at rest, TLS 1.3 protects data in transit, and only an additively homomorphic scheme – Paillier at a 2048-bit modulus – can protect data while it is being computed on, which is the property that distinguishes this design from conventional analytics platforms. JWT-based access control, mutual TLS, and a signed hash-chained audit log together provide the authentication and integrity controls needed to make the resulting computations trustworthy and auditable.